

Date: 18/03/2026
Name: Dhanya S

I am Dhanya S from Kakunje Software working as an Intern. Today I had my 39 session it was an interesting session and mam had taught us about MongoDB and NodeJS. Today I learned about the controllers, how to import and export the file in controllers and performed CRUD operations on it, passing json documents as keys and values and storing that in mongodb compass, and also we had review of our project that is bookstore website.

✓ OPEN EDITORS

✕ JS db.js config

✓ PROJECT

✓ config

JS db.js

✓ controllers

JS customerController.js

JS foodController.js

JS orderController.js

✓ models

JS Customer.js

JS Food.js

JS Order.js

> node_modules

✓ routes

JS customerRoutes.js

JS foodRoutes.js

JS orderRoutes.js

⚙ .env

{ } package-lock.json

{ } package.json

JS server.js

```
config > JS db.js > ...
1  import mongoose from "mongoose";
2
3  const connectDB = async () => {
4    try {
5      await mongoose.connect(process.env.MONGO_URI);
6      console.log("MongoDB Connected");
7    } catch (error) {
8      console.log(error);
9      process.exit(1);
10   }
11 };
12 export default connectDB;
```

controllers > JS customerController.js > deleteCustomer

```
1  import Customer from "../models/Customer.js";
2
3  export const createCustomer = async (req, res) => {
4    try {
5      const customer = await Customer.create(req.body);
6      res.status(201).json(customer);
7    } catch (error) {
8      res.status(500).json({ message: error.message });
9    }
10 };
11
12 export const getCustomers = async (req, res) => {
13   try {
14     const customers = await Customer.find();
15     res.json(customers);
16   } catch (error) {
17     res.status(500).json({ message: error.message });
18   }
19 };
20
21 export const getCustomerById = async (req, res) => {
22   try {
23     const customer = await Customer.findById(req.params.id);
24     if (!customer) {
25       return res.status(404).json({ message: "Customer not found" });
26     }
27     res.json(customer);
28   } catch (error) {
29     res.status(500).json({ message: error.message });
30   }
31 };
32
```

```
32
33 export const updateCustomer = async (req, res) => {
34   try {
35     const customer = await Customer.findByIdAndUpdate(
36       req.params.id,
37       req.body,
38       { new: true }
39     );
40     if (!customer) {
41       return res.status(404).json({ message: "Customer not found" });
42     }
43     res.json(customer);
44   } catch (error) {
45     res.status(500).json({ message: error.message });
46   }
47 };
48
49 export const deleteCustomer = async (req, res) => {
50   try {
51     const customer = await Customer.findByIdAndDelete(req.params.id);
52     if (!customer) {
53       return res.status(404).json({ message: "Customer not found" });
54     }
55     res.json({ message: "Customer deleted successfully" });
56   } catch (error) {
57     res.status(500).json({ message: error.message });
58   }
59 };
```

controllers > JS foodController.js > deleteFood

```
1  import Food from "../models/Food.js";
2
3  export const createFood = async (req, res) => {
4    try {
5      const food = await Food.create(req.body);
6      res.status(201).json(food);
7    } catch (error) {
8      res.status(500).json({ message: error.message });
9    }
10 };
11
12 export const getFoods = async (req, res) => {
13   try {
14     const foods = await Food.find();
15     res.json(foods);
16   } catch (error) {
17     res.status(500).json({ message: error.message });
18   }
19 };
20
21 export const updateFood = async (req, res) => {
22   try {
23     const food = await Food.findByIdAndUpdate(
24       req.params.id,
25       req.body,
26       { new: true }
27     );
28     if (!food) {
29       return res.status(404).json({ message: "Food not found" });
30     }
31     res.json(food);

```

```
30     }
31     res.json(food);
32   } catch (error) {
33     res.status(500).json({ message: error.message });
34   }
35 };
36
37 export const deleteFood = async (req, res) => {
38   try {
39     const food = await Food.findByIdAndDelete(req.params.id);
40     if (!food) {
41       return res.status(404).json({ message: "Food not found" });
42     }
43     res.json({ message: "Food deleted successfully" });
44   } catch (error) {
45     res.status(500).json({ message: error.message });
46   }
47 };
```

controllers > JS orderController.js > ...

```
1  import Order from "../models/Order.js";
2  import Food from "../models/Food.js";
3
4  export const createOrder = async (req, res) => {
5    try {
6      const { customerId, foodId, quantity } = req.body;
7      const food = await Food.findById(foodId);
8      if (!food) {
9        return res.status(404).json({ message: "Food not found" });
10     }
11     const totalPrice = food.price * quantity;
12     const order = await Order.create({
13       customerId,
14       foodId,
15       quantity,
16       totalPrice
17     });
18     res.status(201).json(order);
19   } catch (error) {
20     res.status(500).json({ message: error.message });
21   }
22 };
23
24 export const getOrders = async (req, res) => {
25   try {
26     const orders = await Order.find()
27       .populate("customerId")
28       .populate("foodId");
29     res.json(orders);
30   } catch (error) {
31     res.status(500).json({ message: error.message });
32   }
33 }
```

```
31     res.status(500).json({ message: error.message });
32   }
33 };
34
35 export const getOrderById = async (req, res) => {
36   try {
37     const order = await Order.findById(req.params.id)
38       .populate("customerId")
39       .populate("foodId");
40     if (!order) {
41       return res.status(404).json({ message: "Order not found" });
42     }
43     res.json(order);
44   } catch (error) {
45     res.status(500).json({ message: error.message });
46   }
47 };
48
49 export const updateOrder = async (req, res) => {
50   try {
51     const order = await Order.findByIdAndUpdate(
52       req.params.id,
53       { status: req.body.status },
54       { new: true }
55     );
56     if (!order) {
57       return res.status(404).json({ message: "Order not found" });
58     }
59     res.json(order);
60   } catch (error) {
```



```
59     res.json(order);
60   } catch (error) {
61     res.status(500).json({ message: error.message });
62   }
63 };
64
65 export const deleteOrder = async (req, res) => {
66   try {
67     const order = await Order.findByIdAndDelete(req.params.id);
68     if (!order) {
69       return res.status(404).json({ message: "Order not found" });
70     }
71     res.json({ message: "Order deleted successfully" });
72   } catch (error) {
73     res.status(500).json({ message: error.message });
74   }
75 };
```

models > JS Customer.js > ...

```
1  import mongoose from "mongoose";
2
3  const customerSchema = new mongoose.Schema({
4    name: String,
5    email: String,
6    phone: String,
7    address: String
8  }, { timestamps: true });
9  export default mongoose.model("Customer", customerSchema);
```

models > JS Food.js > ...

```
1  import mongoose from "mongoose";
2
3  const foodSchema = new mongoose.Schema({
4    name: String,
5    price: Number,
6    category: String,
7    isAvailable: {
8      type: Boolean,
9      default: true
10   }
11 }, { timestamps: true });
12 export default mongoose.model("Food", foodSchema);
```

models > JS Order.js > [🔍] default

```
1  import mongoose from "mongoose";
2
3  const orderSchema = new mongoose.Schema({
4    customerId: {
5      type: mongoose.Schema.Types.ObjectId,
6      ref: "Customer"
7    },
8    foodId: {
9      type: mongoose.Schema.Types.ObjectId,
10     ref: "Food"
11   },
12   quantity: Number,
13   totalPrice: Number,
14   status: {
15     type: String,
16     enum: ["pending", "preparing", "delivered"],
17     default: "pending"
18   }
19 }, { timestamps: true });
20 export default mongoose.model("Order", orderSchema);
```

routes > JS customerRoutes.js > ...

```
1  import express from "express";
2  import { createCustomer, getCustomers, getCustomerById, updateCustomer, deleteCustomer } from "../controllers/customerController.js";
3
4  const router = express.Router();
5  router.post("/", createCustomer);
6  router.get("/", getCustomers);
7  router.get("/:id", getCustomerById);
8  router.put("/:id", updateCustomer);
9  router.delete("/:id", deleteCustomer);
10 export default router;
```

routes > JS foodRoutes.js > ...

```
1  import express from "express";
2  import { createFood, getFoods, updateFood, deleteFood } from "../controllers/foodController.js";
3
4  const router = express.Router();
5  router.post("/", createFood);
6  router.get("/", getFoods);
7  router.put("/:id", updateFood);
8  router.delete("/:id", deleteFood);
9  export default router;
```

routes > JS orderRoutes.js > default

```
1  import express from "express";
2  import { createOrder, getOrders, getOrderById, updateOrder, deleteOrder } from "../controllers/orderController.js";
3
4  const router = express.Router();
5  router.post("/", createOrder);
6  router.get("/", getOrders);
7  router.get("/:id", getOrderById);
8  router.put("/:id", updateOrder);
9  router.delete("/:id", deleteOrder);
10 export default router;
```

```
{ } package.json > abc type
```

```
1  {
2    "name": "project",
3    "version": "1.0.0",
4    "description": "",
5    "main": "server.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1",
8      "start": "node server.js"
9    },
10   "keywords": [],
11   "author": "",
12   "license": "ISC",
13   "type": "module",
14   "dependencies": {
15     "dotenv": "^17.3.1",
16     "express": "^5.2.1",
17     "mongoose": "^9.3.1"
18   }
19 }
20
```


JS server.js > ...

```
1  import express from "express";
2  import dotenv from "dotenv";
3  import connectDB from "../config/db.js";
4  import customerRoutes from "../routes/customerRoutes.js";
5  import foodRoutes from "../routes/foodRoutes.js";
6  import orderRoutes from "../routes/orderRoutes.js";
7
8  dotenv.config();
9  connectDB();
10 const app = express();
11 app.use(express.json());
12 app.use("/api/customers", customerRoutes);
13 app.use("/api/foods", foodRoutes);
14 app.use("/api/orders", orderRoutes);
15 const PORT = process.env.PORT;
16 app.listen(PORT, () => {
17   console.log(`Server running on port ${PORT}`);
18 });
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

POSTMAN CONSOLE

○ Node.js v24.13.1

PS C:\Users\Dhanya\OneDrive\Desktop\project> node server.js

[dotenv@17.3.1] injecting env (2) from .env -- tip:  prevent committing .env to

Server running on port 5000

MongoDB Connected

█

⚙ .env

Import to Postman

```
1  PORT = 5000
2  MONGO_URI = mongodb://localhost:27017/Customerscollection
3
4  PORT = 5000
5  MONGO_URI = mongodb://localhost:27017/Foodscollection
6
7  PORT = 5000
8  MONGO_URI = mongodb://localhost:27017/Orderscollection
```

 http://localhost:5000/api/customers

Save | No Environment

POST | http://localhost:5000/api/customers

Send

Params

Authorization

Headers (9)

Body

Scripts

Settings

Code

Cookies

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON

Beautify

```
1 {
2   "name": "Dhanya",
3   "email": "dhanya@gmail.com",
4   "phone": "9876543210",
5   "address": "Udupi"
6 }
```

Body

Cookies

Headers (7)

Test Results

Status: 201 Created | Time: 192 ms | Size: 443 B | ...

Pretty

Raw

Preview

JSON

⌵

⌵

```
1 { "name": "Dhanya", "email": "dhanya@gmail.com", "phone": "9876543210", "address": "Udupi", "_id": "69ba9b59c76cc2771a884433", "createdAt": "2026-03-18T12:32:25.084Z", "updatedAt": "2026-03-18T12:32:25.084Z", "_v": 0 }
```

http://localhost:5000/api/customers

Save

No Environment

GET

http://localhost:5000/api/customers

Send

Params

Authorization

Headers (7)

Body

Scripts

Settings

Code

Cookies

☒ none

☐ form-data

☐ x-www-form-urlencoded

☐ raw

☐ binary

☐ GraphQL

This request does not have a body

Body

Cookies

Headers (7)

Test Results

Status: 200 OK

Time: 148 ms

Size: 440 B

⋮

⌕

⌵

Pretty

Raw

Preview

JSON

⌵

↻

1

[{"_id":"69ba9b59c76cc2771a884433","name":"Dhanya","email":"dhanya@gmail.com","phone":"9876543210","address":"Udupi","createdAt":"2026-03-18T12:32:25.084Z","updatedAt":"2026-03-18T12:32:25.084Z","__v":0}]

 http://localhost:5000/api/customers

Save

No Environment

GET http://localhost:5000/api/customers

Send

ParamsAuthorizationHeaders (7)BodyScriptsSettings

CodeCookies

BodyCookiesHeaders (7)Test Results

Status: 200 OK Time: 14 ms Size: 644 B

PrettyRawPreviewJSON

```
1 [
2   {
3     "_id": "69ba9b59c76cc2771a884433",
4     "name": "Dhanya",
5     "email": "dhanya@gmail.com",
6     "phone": "9876543210",
7     "address": "Udupi",
8     "createdAt": "2026-03-18T12:32:25.084Z",
9     "updatedAt": "2026-03-18T12:32:25.084Z",
10    "__v": 0
11  },
12  {
13    "_id": "69ba9d02c76cc2771a884436",
14    "name": "Diya",
15    "email": "diya@gmail.com",
16    "phone": "1234567890",
17    "address": "Mangalore",
18    "createdAt": "2026-03-18T12:39:30.316Z",
19    "updatedAt": "2026-03-18T12:39:30.316Z",
20    "__v": 0
21  }
22 ]
```

 http://localhost:5000/api/customers/69ba9d02c76cc2771a884436

 Save 

No Environment 

GET  http://localhost:5000/api/customers/69ba9d02c76cc2771a884436

Send 

Params Authorization Headers (7) Body Scripts Settings [Code](#) [Cookies](#)

Body Cookies Headers (7) Test Results  Status: 200 OK Time: 61 ms Size: 438 B   

Pretty Raw Preview JSON 

```
1 {
2   "id": "69ba9d02c76cc2771a884436",
3   "name": "Diya",
4   "email": "diya@gmail.com",
5   "phone": "1234567890",
6   "address": "Mangalore",
7   "createdAt": "2026-03-18T12:39:30.316Z",
8   "updatedAt": "2026-03-18T12:39:30.316Z",
9   "__v": 0
10 }
```

HTTP

http://localhost:5000/api/customers/69ba9b59c76cc2771a884433

Save

No Environment

Send

PUT

http://localhost:5000/api/customers/69ba9b59c76cc2771a884433

Params

Authorization

Headers (9)

Body

Scripts

Settings

Code

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Beautify

1

{

2

"name": "Dhanya S",

3

"phone": "8888888888"

4

}

Body

Cookies

Headers (7)

Test Results

Status: 200 OK Time: 80 ms Size: 440 B

Pretty

Raw

Preview

JSON

1

{

2

"_id": "69ba9b59c76cc2771a884433",

3

"name": "Dhanya S",

4

"email": "dhanya@gmail.com",

5

"phone": "8888888888",

6

"address": "Udupi",

7

"createdAt": "2026-03-18T12:32:25.084Z",

8

"updatedAt": "2026-03-18T12:43:09.198Z",

9

"__v": 0

10

}

 http://localhost:5000/api/customers/69ba9b59c76cc2771a884433

Save

No Environment

DELETE

http://localhost:5000/api/customers/69ba9b59c76cc2771a884433

Send

Params

Authorization

Headers (7)

Body

Scripts

Settings

Code

Cookies

☒ none

☐ form-data

☐ x-www-form-urlencoded

☐ raw

☐ binary

☐ GraphQL

This request does not have a body

Body

Cookies

Headers (7)

Test Results

Status: 200 OK

Time: 19 ms

Size: 278 B

...

...

...

Pretty

Raw

Preview

JSON

...

1

2

3

{

"message": "Customer deleted successfully"

}

Type a query: { field: 'value' } or [Generate query](#)

Explain Reset Find Options

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE

25 1 - 2 of 2

```
_id: ObjectId('69ba9b59c76cc2771a884433')
name: "Dhanya"
email: "dhanya@gmail.com"
phone: "9876543210"
address: "Udupi"
createdAt: 2026-03-18T12:32:25.004+00:00
updatedAt: 2026-03-18T12:32:25.004+00:00
__v: 0
```

```
_id: ObjectId('69ba9d02c76cc2771a884436')
name: "Diya"
email: "diya@gmail.com"
phone: "1234567890"
address: "Mangalore"
createdAt: 2026-03-18T12:39:30.316+00:00
updatedAt: 2026-03-18T12:39:30.316+00:00
__v: 0
```

>_ Open MongoDB shell

Documents 1

Schema

Validation

Explain

Find

Options ▶

 UPDATE

EXPORT DATA ▼

EXPORT CODE

1-1 of 1

< >



```
_id: ObjectId('69ba9d02c76cc2771a884436')
name: "Diya"
email: "diya@gmail.com"
phone: "1234567890"
address: "Mangalore"
createdAt: 2026-03-18T12:39:30.316+00:00
updatedAt: 2026-03-18T12:39:30.316+00:00
__v: 0
```

POST http://localhost:5000/api/foods

Send

Params

Authorization

Headers (9)

Body

Scripts

Settings

Code

Cookies

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON

Beautify

```
1 {
2   "name": "Burger",
3   "price": 150,
4   "category": "Fast Food",
5   "isAvailable": true
6 }
```

Body

Cookies

Headers (7)

Test Results

Status: 201 Created Time: 97 ms Size: 431 B

Pretty

Raw

Preview

JSON

```
1 {
2   "name": "Burger",
3   "price": 150,
4   "category": "Fast Food",
5   "isAvailable": true,
6   "_id": "69ba9ebfc76cc2771a88443d",
7   "createdAt": "2026-03-18T12:46:55.914Z",
8   "updatedAt": "2026-03-18T12:46:55.914Z",
9   "__v": 0
10 }
```

HTTP

http://localhost:5000/api/foods

Save

No Environment

Send

POST

http://localhost:5000/api/foods

Params

Authorization

Headers (9)

Body

Scripts

Settings

Code

Cookies

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON

Beautify

```
1 {
2   "name": "Veg Biryani",
3   "price": 200,
4   "category": "Main Course",
5   "isAvailable": true
6 }
```

Body

Cookies

Headers (7)

Test Results

Status: 201 Created Time: 25 ms Size: 438 B

Pretty

Raw

Preview

JSON

```
1 {
2   "name": "Veg Biryani",
3   "price": 200,
4   "category": "Main Course",
5   "isAvailable": true,
6   "_id": "69ba9eeac76cc2771a88443f",
7   "createdAt": "2026-03-18T12:47:38.131Z",
8   "updatedAt": "2026-03-18T12:47:38.131Z",
9   "__v": 0
10 }
```

HTTP

http://localhost:5000/api/foods

Save

No Environment

Send

GET

http://localhost:5000/api/foods

Params

Authorization

Headers (7)

Body

Scripts

Settings

Body

Cookies

Headers (7)

Test Results

Status: 200 OK

Time: 32 ms

Size: 627 B

Code

Cookies

Pretty

Raw

Preview

JSON

```
1 [
2   {
3     "_id": "69ba9ebfc76cc2771a88443d",
4     "name": "Burger",
5     "price": 150,
6     "category": "Fast Food",
7     "isAvailable": true,
8     "createdAt": "2026-03-18T12:46:55.914Z",
9     "updatedAt": "2026-03-18T12:46:55.914Z",
10    "__v": 0
11  },
12  {
13    "_id": "69ba9eeac76cc2771a88443f",
14    "name": "Veg Biryani",
15    "price": 200,
16    "category": "Main Course",
17    "isAvailable": true,
18    "createdAt": "2026-03-18T12:47:38.131Z",
19    "updatedAt": "2026-03-18T12:47:38.131Z",
20    "__v": 0
21  }
22 ]
```

http://localhost:5000/api/foods/69ba9ebfc76cc2771a88443d

Save

No Environment

PUT

http://localhost:5000/api/foods/69ba9ebfc76cc2771a88443d

Send

Params

Authorization

Headers (9)

Body

Scripts

Settings

Code

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Beautify

1

2

3

4

{

"name": "pizza",

"price": 350

}

Body

Cookies

Headers (7)

Test Results

Status: 200 OK

Time: 32 ms

Size: 425 B

More

Fullscreen

Search

Pretty

Raw

Preview

JSON

Copy

1

2

3

4

5

6

7

8

9

10

{

"_id": "69ba9ebfc76cc2771a88443d",

"name": "pizza",

"price": 350,

"category": "Fast Food",

"isAvailable": true,

"createdAt": "2026-03-18T12:46:55.914Z",

"updatedAt": "2026-03-18T12:50:57.234Z",

"__v": 0

}

 http://localhost:5000/api/foods/69ba9ebfc76cc2771a88443d

Save

No Environment

DELETE

http://localhost:5000/api/foods/69ba9ebfc76cc2771a88443d

Send

Params

Authorization

Headers (7)

Body

Scripts

Settings

Code

Cookies

☒ none

☐ form-data

☐ x-www-form-urlencoded

☐ raw

☐ binary

☐ GraphQL

This request does not have a body

Body

Cookies

Headers (7)

Test Results

Status: 200 OK

Time: 18 ms

Size: 274 B

Pretty

Raw

Preview

JSON

1

2

3

```
[{"message": "Food deleted successfully"}]
```

Documents 0 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

ExplainResetFindOptions

ADD DATAUPDATEDELETEEXPORT DATAEXPORT CODE

251 - 1 of 1

```
_id: ObjectId('69ba9eeac76cc2771a88443f')
name: "Veg Biryani"
price: 200
category: "Main Course"
isAvailable: true
createdAt: 2026-03-18T12:47:38.131+00:00
updatedAt: 2026-03-18T12:47:38.131+00:00
__v: 0
```


 http://localhost:5000/api/orders

Save | No Environment

POST | http://localhost:5000/api/orders | Send

Params | Authorization | Headers (9) | Body | Scripts | Settings

Code | Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

Beautify

```
1 {
2   "customerId": "69ba9d02c76cc2771a884436",
3   "foodId": "69ba9eeac76cc2771a88443f",
4   "quantity": 2
5 }
```

Body | Cookies | Headers (7) | Test Results

Status: 201 Created | Time: 432 ms | Size: 486 B

Pretty | Raw | Preview | JSON

```
1 [{"customerId":"69ba9d02c76cc2771a884436","foodId":"69ba9eeac76cc2771a88443f","quantity":2,"totalPrice":400,"status":"pending",
  "_id":"69baa0b3c76cc2771a884446","createdAt":"2026-03-18T12:55:15.667Z","updatedAt":"2026-03-18T12:55:15.667Z","__v":0}]
```

HTTP

http://localhost:5000/api/orders

Save

No Environment

Send

GET

http://localhost:5000/api/orders

Params

Authorization

Headers (7)

Body

Scripts

Settings

Code

Cookies

Query Params

Key	Value
-----	-------

Body

Cookies

Headers (7)

Test Results

Status: 200 OK Time: 21 ms Size: 831 B

Pretty

Raw

Preview

JSON

```
1 {
2   "_id": "69baa0b3c76cc2771a884446",
3   "customerId": {
4     "_id": "69ba9d02c76cc2771a884436",
5     "name": "Diya",
6     "email": "diya@gmail.com",
7     "phone": "1234567890",
8     "address": "Mangalore",
9     "createdAt": "2026-03-18T12:39:30.316Z",
10    "updatedAt": "2026-03-18T12:39:30.316Z",
11    "__v": 0
12  },
13  "foodId": {
14    "_id": "69ba9eeac76cc2771a88443f",
15    "name": "Veg Biryani",
16    "price": 200,
17    "category": "Main Course",
18    "isAvailable": true,
19    "createdAt": "2026-03-18T12:47:38.131Z",
20    "updatedAt": "2026-03-18T12:47:38.131Z",
21    "__v": 0
22  }
```

http://localhost:5000/api/orders

Save

No Environment

GET

http://localhost:5000/api/orders

Send

Params

Authorization

Headers (7)

Body

Scripts

Settings

Code

Cookies

Query Params

Key	Value
-----	-------

Body

Cookies

Headers (7)

Test Results

Status: 200 OK Time: 21 ms Size: 831 B

Pretty

Raw

Preview

JSON

```
10    "createdAt": "2026-03-18T12:39:30.316Z",
11    "updatedAt": "2026-03-18T12:39:30.316Z",
12    "__v": 0
13  },
14  "foodId": {
15    "_id": "69ba9eeac76cc2771a88443f",
16    "name": "Veg Biryani",
17    "price": 200,
18    "category": "Main Course",
19    "isAvailable": true,
20    "createdAt": "2026-03-18T12:47:38.131Z",
21    "updatedAt": "2026-03-18T12:47:38.131Z",
22    "__v": 0
23  },
24  "quantity": 2,
25  "totalPrice": 400,
26  "status": "pending",
27  "createdAt": "2026-03-18T12:55:15.667Z",
28  "updatedAt": "2026-03-18T12:55:15.667Z",
29  "__v": 0
30 }
31 }
```

http://localhost:5000/api/orders/69baa0b3c76cc2771a884446

Save No Environment

GET http://localhost:5000/api/orders/69baa0b3c76cc2771a884446 Send

Params Authorization Headers (7) Body Scripts Settings Code Cookies

Query Params	
Key	Value

Body Cookies Headers (7) Test Results Status: 200 OK Time: 26 ms Size: 829 B

Pretty Raw Preview JSON

```
1 {
2   "_id": "69baa0b3c76cc2771a884446",
3   "customerId": {
4     "_id": "69ba9d02c76cc2771a884436",
5     "name": "Diya",
6     "email": "diya@gmail.com",
7     "phone": "1234567890",
8     "address": "Mangalore",
9     "createdAt": "2026-03-18T12:39:30.316Z",
10    "updatedAt": "2026-03-18T12:39:30.316Z",
11    "__v": 0
12  },
13  "foodId": {
14    "_id": "69ba9eeac76cc2771a88443f",
15    "name": "Veg Biryani",
16    "price": 200,
17    "category": "Main Course",
18    "isAvailable": true,
19    "createdAt": "2026-03-18T12:47:38.131Z",
20    "updatedAt": "2026-03-18T12:47:38.131Z",
21    "__v": 0
22  },
23 }
```

PUT

http://localhost:5000/api/orders/69baa0b3c76cc2771a884446

Send

Params

Authorization

Headers (9)

Body

Scripts

Settings

Code

Cookies

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON

Beautify

```
1 {
2   "status": "preparing"
3 }
```

Body

Cookies

Headers (7)

Test Results

Status: 200 OK

Time: 50 ms

Size: 483 B

...

...

...

Pretty

Raw

Preview

JSON

...

...

...

```
1 {
2   "_id": "69baa0b3c76cc2771a884446",
3   "customerId": "69ba9d02c76cc2771a884436",
4   "foodId": "69ba9eeac76cc2771a88443f",
5   "quantity": 2,
6   "totalPrice": 400,
7   "status": "preparing",
8   "createdAt": "2026-03-18T12:55:15.667Z",
9   "updatedAt": "2026-03-18T12:59:45.512Z",
10  "__v": 0
11 }
```

customers foods orders +

Customerscollection > Orderscollection > orders

Open MongoDB shell

Documents 0 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain

Reset

Find

Options

ADD DATA

UPDATE

DELETE

EXPORT DATA

EXPORT CODE

25

1 of 1



```
_id: ObjectId('69baa8b3c76cc2771a884446')
customerId: ObjectId('69ba9d82c76cc2771a884436')
foodId: ObjectId('69ba9eeac76cc2771a88443f')
quantity: 2
totalPrice: 400
status: "preparing"
createdAt: 2026-03-18T12:55:15.667+00:00
updatedAt: 2026-03-18T12:59:45.512+00:00
__v: 0
```



 http://localhost:5000/api/orders/69baa0b3c76cc2771a884446

Save | No Environment

DELETE | http://localhost:5000/api/orders/69baa0b3c76cc2771a884446 | Send

Params | Authorization | Headers (7) | Body | Scripts | Settings

Code | Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body | Cookies | Headers (7) | Test Results

🌐 Status: 200 OK Time: 27 ms Size: 275 B ...

Pretty | Raw | Preview | JSON |

```
1 {
2   "message": "Order deleted successfully"
3 }
```